CAMPUS RAG / SETUP GUIDE

校园 RAG 知识库平台
详细安装与配置指南

Windows x64 / Java 17 / Python 3.11 / Docker Linux

从安装工具到环境验收，一份可以逐步执行的说明

真实仓库与路径 · 软件安装 · Docker 客户端适配

SDK 与模型准备 · 服务连接 · 验收及故障排查

项目仓库

https://github.com/ZhiShan717/campus-rag-platform

负责人现有目录：D:\campus_RAG_work

新成员示例目录：D:\Projects\campus-rag-platform

当前交付边界

环境、基础服务和模型可准备；业务应用仍为目录骨架。

编制日期 2026-10-23 | 基线 ee25619 | 项目共享版

依据原始 7 页手册与当前仓库整理；不包含真实密码、令牌或个人授权。

阅读导航

安装前先阅读使用范围与实际路径。首次安装从第 04 节继续；已有环境按角色跳转，避免重复初始化。目录和右侧页码可点击。

01 使用范围与完成目标 . . . 3
02 仓库、目录与连接入口 . . . 4
03 项目结构与资料定位 . . . 5
04 版本、硬件与安装顺序 . . . 6
05 Windows、虚拟化与 WSL2 . . . 7
06 Git、VS Code 与首次克隆 . . . 8
07 JDK 17 与 Maven . . . 9
08 Python、Node.js 与虚拟环境 . . . 10
09 Docker Desktop 安装与客户端适配 . . . 11
10 SDK 安装与本机参数初始化 . . . 12
11 新队员启动五项或七项基础服务 . . . 13
12 服务地址、数据库客户端与数据卷 . . . 14
13 Ollama、嵌入模型与重排模型安装 . . . 15
14 模型配置、依赖锁与内存控制 . . . 16
15 工具、基础服务与模型分层验收 . . . 17
16 Java、Python 与前端当前可做的配置 . . . 18
17 日常启动、停止与负责人本机复用 . . . 19
18 业务工程实现后的最小联调顺序 . . . 20
19 RAG、引用、版本与权限验收标准 . . . 21
20 环境与安装问题排查 . . . 22
21 业务链路问题与团队协作 . . . 23
22 原手册修订对照 . . . 24
23 官方安装资料与参考 . . . 25

01 使用范围与完成目标

本指南供项目负责人和新队员逐步配置开发环境。项目采用 Java 微服务、Python RAG 运行时与 Vue 前端；目标链路是文档上传、对象存储、解析分块、向量入库、混合检索、版本与权限过滤、重排，以及带引用的回答。

当前可完成：开发工具安装，仓库克隆，Python SDK 安装，五项或七项中间件启动，中文分词和认证检查，三类模型准备，以及 Maven 聚合结构验证。

当前不能完成：直接启动业务网站、Java 业务服务、Python API、前端页面或完整问答链路。仓库仍为目录骨架，没有 Java 启动类、Python API 入口、前端 package.json、业务表及 ES 索引定义。

路线	适用对象	本指南中的操作
新队员首次安装	新电脑或新克隆	按 02-14 节执行；Docker 优先采用 09 节，复用本人已有 Desktop
负责人已有环境	D:\campus_RAG_work 已配置电脑	不重装、不重建虚拟环境、不重置凭据；重点看 15-17 节
业务开发与联调	模块工程实现后	18-19 节是后续建设与验收标准，不是当前已实现功能

全文命令默认在对应仓库根目录的 PowerShell 执行。出现“管理员 PowerShell”时才使用管理员窗口。每个代码块按顺序执行，遇到错误先停下排查，不应跳过失败步骤继续启动。

新队员 Docker 路线来自对脚本依赖的审核，尚未在另一台全新电脑完成端到端安装验收。请按实际安装路径和 Docker 状态核对，不能把克隆等同于自动安装。

02 仓库、目录与连接入口

信息	实际值
GitHub 项目主页	https://github.com/ZhiShan717/campus-rag-platform
HTTPS 克隆地址	https://github.com/ZhiShan717/campus-rag-platform.git
默认分支与负责人	main；GitHub 用户 ZhiShan717
仓库可见性	公开，读取和克隆无需邀请；向原仓库推送需写入权限
协作者管理入口	https://github.com/ZhiShan717/campus-rag-platform/settings/access
负责人现有工作目录	D:\campus_RAG_work
新队员示例工作目录	D:\Projects\campus-rag-platform，可换为自己的路径
本指南 PDF	<仓库根目录>\output\pdf\校园RAG平台_详细安装与配置指南.pdf
本指南可编辑正文	<仓库根目录>\docs\team\详细安装与配置指南.md

先确定自己的仓库根目录。例如新队员使用 D:\Projects\campus-rag-platform；负责人直接使用已有的 D:\campus_RAG_work，不要在原目录中再次克隆或重新初始化 Git。

```
# 新队员完成克隆后，进入自己的项目目录
Set-Location -LiteralPath 'D:\Projects\campus-rag-platform'
Get-Location
git remote -v
git branch --show-current
```

成功标准：当前目录包含 README.md、pom.xml、backend、environment；origin 指向上述克隆地址，工作分支按团队约定使用。

当前没有团队共享服务器、统一 LLM API 或业务网址。127.0.0.1 指使用者自己的电脑，队员不能用它连接负责人电脑。密码由各自初始化生成，不在本文中给出。

03 项目结构与资料定位

```
campus-rag-platform/
  pom.xml                Java 17 聚合构建入口
  backend/
    gateway-service/     网关与路由
    auth-service/        用户、角色与权限
    doc-service/         文档、版本与处理任务
    search-service/      检索及 Java/Python 协作
    admin-service/       管理功能
    eval-service/        评测任务与结果
  rag-runtime/           Python RAG、编排、MCP、护栏、评测
  frontend/              Vue 页面、组件与 API 调用
  database/              迁移、样例与索引定义
  deploy/                后续业务 Docker/Nginx/K8s 部署
  tests/                 跨模块 integration/e2e/fixtures
  docs/                  团队、开发、接口、架构说明
  environment/           开发环境脚本、Compose、依赖锁、原手册
  output/pdf/            本项目安装指南 PDF
  .local/                本机工具、模型与 Docker 配置/数据
  .venv/                 本机 Python 虚拟环境
  .setup/                本机状态、日志与检查记录
```

要查的资料	仓库相对路径
新队员速查及完整协作步骤	docs/team/队员信息速查.md；docs/team/团队协作与接入.md
文件归属和开发约定	docs/development/项目目录与开发约定.md
当前开发环境说明	environment/README.md
开发中间件配置	environment/docker-compose.yml
SDK 与模型依赖锁	environment/requirements-rag.lock.txt；requirements-models.lock.txt
无密钥模型参数	environment/model-env.example
原始参考手册	environment/docs/校园级RAG知识库平台环境安装与首次运行手册.pdf

环境配置放 environment，业务部署放 deploy；不要把中间件 Compose 误认成业务应用部署。个人登录助手、负责人一键包装、编辑器工作区、AGENTS.md 只留本机，不随 Git 克隆；安装过程不要依赖这些文件。

04 版本、硬件与安装顺序

范围	项目记录基线 / 约定
Java / Maven	Java 17；Maven 3.9.9，根 POM 目前未引入 Spring 依赖
Python	3.11 x64；负责人已验证 3.11.9，模型 wheel 固定 cp311-win_amd64
Node / npm	负责人记录 Node.js 24.22.0 / npm 11.19.1；前端工具链未初始化
Python SDK	LangChain 1.4.7、LangGraph 1.2.16、MCP 2.3.0、FastAPI 0.159.1
CPU 模型依赖	PyTorch 2.14.1+cpu、Sentence Transformers 6.2.0、Transformers 5.20.2
Docker 记录	Desktop 4.96.2、Engine 29.9.2、Compose 5.6.1；须使用 Linux 容器
中间件	MySQL 8.0、Redis 7.4.2、ES+IK 8.11.4、Nacos 2.3.2、RocketMQ 5.3.1
模型	BGE large 中文 1024 维；bge-reranker-base；Ollama qwen3:1.7b

上表用于复现仓库，不代表这些版本是当前官方最新版本。SDK 和模型安装优先使用锁文件，避免按原手册下限重新解析出另一套依赖。Python 3.11.9 是项目已验证且提供传统 Windows 安装器的兼容基线，官方已有后续安全更新。

Docker 官方 WSL 后端要求包含 WSL 2.1.5 或更高版本、至少 8 GB 系统内存、支持 SLAT 的处理器和已启用的硬件虚拟化。Windows 还应满足当前官方支持范围，旧版 Windows 10 不能仅凭版本名称判断受支持。[S01]

本项目的资源规划建议：16 GB 可分批运行工具、服务和 CPU 模型；32 GB 更便于完整联调。建议预留约 40 GB 或更多空闲空间供镜像、模型与缓存增长，实际用量取决于下载内容。这是项目预算建议，不是 Docker 官方最低要求。

安装顺序：Windows/WSL2 → Git/VS Code → JDK/Maven → Python/Node → Docker/Ollama → 克隆仓库 → 路径和 Docker 客户端适配 → SDK → 中间件 → 模型 → 分层验收。

05 Windows、虚拟化与 WSL2

1. 在任务管理器的“性能 / CPU”查看“虚拟化”是否已启用。未启用时，在 BIOS/UEFI 开启 Intel VT-x 或 AMD-V；具体入口按电脑厂商说明。
2. 使用仍受支持的 Windows 版本并安装必要更新。以管理员身份打开 PowerShell，为 Docker 准备 WSL；下面的方式不额外安装 Ubuntu。[S02]

```
# 管理员 PowerShell：首次启用 WSL
wsl --install --no-distribution
```

3. 按提示重启电脑。重新打开管理员 PowerShell，更新 WSL 并将新发行版默认设为版本 2。[S03]

```
wsl --update
wsl --set-default-version 2
wsl --version
wsl --status
wsl -l -v
```

成功标准：WSL 版本满足 Docker 当前要求；系统虚拟化可用。只为 Docker 使用 WSL2 时，不要求额外创建个人 Ubuntu 发行版，列表中没有 Ubuntu 不等于安装失败。

如果旧系统不识别 wsl --install 或 --version，先按微软文档完成系统 / WSL 更新。公司或学校设备的功能安装被策略限制时，应由设备管理员按策略处理。

此阶段不需要安装独立的 MySQL、Redis、ES 或 Nacos Windows 服务；后续由项目 Compose 提供，避免重复安装占用端口。

06 Git、VS Code 与首次克隆

1. 从 Git 官方 Windows 页面下载适合本机的安装器。[S04] 选择允许从命令行及第三方软件使用 Git；安装完重新打开 PowerShell，执行 git --version。
2. 从 VS Code 官方页面安装 Windows User Setup，勾选加入 PATH，重新开终端执行 code --version。[S05]
3. 设置本人提交身份。示例使用交互输入，避免把负责人的姓名邮箱复制给所有队员。

```
git --version
$gitName = Read-Host 'Your Git commit name'
$gitEmail = Read-Host 'Your Git commit email'
git config --global user.name "$gitName"
git config --global user.email "$gitEmail"
```

4. 仅在尚未克隆时执行以下命令；若目标目录已存在，先确认是否就是已有工作区，不要覆盖。

```
New-Item -ItemType Directory -Path 'D:\Projects' -Force | Out-Null
Set-Location -LiteralPath 'D:\Projects'
git clone https://github.com/ZhiShan717/campus-rag-platform.git
Set-Location -LiteralPath 'D:\Projects\campus-rag-platform'
git remote -v
code .
```

5. 在 VS Code 安装 Java Extension Pack、Python、Pylance；前端成员另装 Vue - Official。全工具检查会检查这些扩展及 GitHub CLI；未承担对应模块时，需结合检查项判断，不应把缺少可选工具等同于中间件启动失败。

公开仓库克隆不要求 GitHub 登录。推送和 Pull Request 使用本人账号；是否拥有写入权限与仓库是否公开是两件事。协作者邀请由负责人在成员管理页面发送，对方接受后生效。

07 JDK 17 与 Maven

1. 从 Eclipse Temurin 官方页面选择 Java 17、Windows、x64、JDK 安装包，避免下载成只有运行时的 JRE。[S06] MSI 的自定义安装选项可启用 Add to PATH 和 Set JAVA_HOME。
2. JAVA_HOME 指向 JDK 根目录，值中不要包含 bin；PATH 添加 %JAVA_HOME%\bin。安装后重新打开终端验证。
3. 从 Maven 官方下载项目约定的 3.9.x Binary zip，已验证基线为 3.9.9；解压到自己的工具目录，将其 bin 加入 PATH。[S07] 不要选择 Source archive。

```
java -version
javac -version
mvn -version
```

成功标准：java 和 javac 均为 17，Maven 能启动且报告使用 Java 17。Maven 只是命令行工具，解压即可，不需要 Windows 服务。

配置	示例值及含义
JAVA_HOME	例如 D:\devtools\jdk-17；使用自己的实际 JDK 根目录
MAVEN_HOME	例如 D:\devtools\apache-maven-3.9.9；仓库开发终端会使用此项
PATH	包含上述 JDK 的 bin 以及 Maven 的 bin；重新打开终端后生效

还需核对项目脚本：environment/Enter-DevEnvironment.ps1 每次运行会将 JAVA_HOME 设为 D:\JDK，将 MAVEN_HOME 设为 D:\maven\apache-maven-3.9.9-bin\apache-maven-3.9.9。若本机路径不同，必须在本机核对并调整对应赋值；只设置系统环境变量仍可能被脚本覆盖。

该脚本还列有 Node、Git、VS Code 的 D 盘路径候选。候选不存在时可继续使用系统 PATH 中的安装，但应以 Get-Command 和工具版本的实际结果确认。纯个人路径调整不混入业务提交；需要团队共享的可移植性改进应单独审查。

08 Python、Node.js 与虚拟环境

1. 安装 Python 3.11 x64，选择完整安装器并保留 pip 与 Python Launcher；不要使用 embeddable zip。Windows 安装说明与 3.11.9 兼容基线下载见 [S08]。
2. 安装时可勾选 Add python.exe to PATH，但后续仍用 py -3.11 明确选择版本，避免电脑上已有 Python 3.12/3.14 时选错。
3. Node.js 使用团队约定的 24.x 基线或后续前端包锁要求；官方安装包包含 npm。[S09] 前端尚无工程入口，安装工具不等于应用可启动。

```
py --list
py -3.11 --version
node --version
npm.cmd --version
```

4. 在新克隆仓库根目录创建一次虚拟环境。负责人已有 .venv 时不要重建，先检查已有解释器。

```
py -3.11 -m venv .venv
.\.venv\Scripts\python.exe --version
.\.venv\Scripts\python.exe -m pip --version
```

成功标准：虚拟环境解释器报告 Python 3.11，pip 路径位于本仓库 .venv。模型 CPU 安装脚本固定 cp311-win_amd64，不能用 Python 3.12 或 ARM64 解释器直接替代。

后续不必激活环境，也能使用完整路径调用 .venv\Scripts\python.exe。PowerShell 若阻止 npm.ps1，使用 npm.cmd；若阻止项目 .ps1 且设备策略允许，可只在当前窗口使用 Set-ExecutionPolicy -Scope Process Bypass，不改全局策略。

09 Docker Desktop 安装与客户端适配

先从 Docker 官方 Windows 入口安装 Desktop，选择 WSL2 后端，打开后确认使用 Linux containers。[S01] 用本人 Windows 开始菜单启动自己的 Docker Desktop，等待引擎就绪。

在普通 PowerShell 中验证 docker version 的 Client 和 Server 都能返回，docker compose version 可用，docker info --format '{{.OSType}}' 返回 linux。安装器退出不等于引擎已经运行。

新队员采用本节路线：复用本人已启动的 Desktop 引擎，只建立项目独立的 CLI 配置，不调用会切换独立数据目录的 Start-ContainerEngine.ps1。下面命令在仓库根目录执行。[S10]

```
$env:DOCKER_CONFIG = Join-Path (Get-Location) '.local\docker-cli'
$env:DOCKER_CONTEXT = $null
$env:DOCKER_HOST = $null
$env:DOCKER_TLS_VERIFY = $null
$env:DOCKER_CERT_PATH = $null
.\environment\Initialize-ContainerClient.ps1

$contexts = @(docker context ls --format '{{.Name}}')
if ($LASTEXITCODE -ne 0) { throw 'Cannot list Docker contexts' }
if ('desktop-linux' -notin $contexts) {
    docker context create desktop-linux `
        --docker 'host=npipe:////./pipe/dockerDesktopLinuxEngine'
    if ($LASTEXITCODE -ne 0) { throw 'Cannot create Docker context' }
}
$env:DOCKER_CONTEXT = 'desktop-linux'
docker context inspect desktop-linux --format '{{.Endpoints.docker.Host}}'
docker info --format '{{.OSType}}'
docker compose version
```

上述 named pipe 是 Windows Docker Desktop Linux 引擎的常见端点；如本机不是该端点或检查失败，应先在本人原 Docker 配置下检查 context inspect 的实际 Host，不要强行替换已存在的 context。成功标准是目标为自己的本机 Linux 引擎，而不是远程服务器或 Windows containers。

插件初始化脚本会寻找仓库内 .local\DockerDesktop\resources\cli-plugins，以及 %LOCALAPPDATA%\Programs\DockerDesktop\resources\cli-plugins、%ProgramFiles%\Docker\Docker\resources\cli-plugins。若 Docker 装在其他位置，需按真实安装路径适配候选目录。

10 SDK 安装与本机参数初始化

完成 Python 虚拟环境后，从仓库根目录安装 SDK。安装脚本使用 SDK 锁文件、运行依赖一致性检查和离线调用检查，不启动业务 API。

```
.\environment\Install-PythonDependencies.ps1
.\.venv\Scripts\python.exe -m pip check
.\.venv\Scripts\python.exe .\environment\Test-PythonDependencies.py
```

成功标准：pip check 无依赖冲突，SDK 检查报告 PASS。优先按 requirements-rag.lock.txt 安装，不执行未经团队确认的整套升级，也不执行原手册的 pip install -e .：当前没有对应 pyproject.toml。

初始化基础服务凭据。该步骤只在本机生成 environment/.env；重复执行保留已有值，不轮换已有数据库中的密码。

```
.\environment\Initialize-Environment.ps1
Test-Path -LiteralPath '.\environment\.env'
```

成功标准：文件存在，脚本报告已有或新建凭据已保护。不要把文件正文输出到群聊、Issue、截图或共享日志。该文件与持久化数据卷配套保管。

文件	内容及维护方式
requirements-rag.in / .lock.txt	SDK 依赖声明与固定安装清单；日常使用锁文件
requirements-models.in / .lock.txt	模型依赖声明与固定清单；安装时同时遵守 SDK 约束
environment/.env	服务随机凭据，只留本机
environment/.env.application	服务连接参数、模型路径与随机 APP/JWT 密钥，后续生成

同一 Docker 引擎的 Compose 项目名固定为 campus-rag-dev。另一份克隆仍可能连接同名数据卷；不要以为换目录就创建了独立空数据库，更不要用新密码覆盖旧卷对应的凭据。

11 新队员启动五项或七项基础服务

本节接着 09 节已连接本机 Linux 引擎的终端执行。加载 Common.ps1 后，将继续使用项目独立 Docker CLI 配置与 desktop-linux；它提供固定项目名、Compose 文件和环境文件的参数。

```
. .\environment\Common.ps1
$composeArgs = @(Get-CampusComposeArguments)
# 需要 RocketMQ 时，上行改为：
# $composeArgs = @(Get-CampusComposeArguments -Messaging)

docker @composeArgs config --quiet
if ($LASTEXITCODE -ne 0) { throw 'Compose validation failed' }
docker @composeArgs up --detach --build --wait --wait-timeout 600
if ($LASTEXITCODE -ne 0) { throw 'Infrastructure startup failed' }
Set-CampusNacosPassword
```

默认启动 MySQL、Redis、Elasticsearch+IK、MinIO、Nacos 五项；Messaging 增加 NameServer 和 Broker 两项长期服务。消息卷权限初始化是一次性辅助容器，不算业务服务。

此方式直接使用共用 Compose，不执行负责人独立 Docker Desktop 数据目录启动器。镜像首次拉取和 IK 构建可能较久；网络失败时先检查官方镜像源连通性或本人 Docker 登录，不修改镜像为未经核实的任意来源。

随后按所选范围验收并生成应用参数：

```
.\environment\Test-Services.ps1
# 若已启用消息组件，改用：
# .\environment\Test-Services.ps1 -Messaging
.\environment\Initialize-ApplicationEnvironment.ps1
docker @composeArgs ps
```

成功标准：所选服务均 healthy；MinIO、Nacos、ES 认证和 IK 分词通过；Messaging 时 Broker 真实注册检查通过。Nacos 使用独立 Derby 卷，不要求先创建 campus_rag 业务表。

业务连接参数在 environment/.env.application，实际应用仍需实现变量映射和配置加载。初始化文件本身不会创建 Java/Python 服务，也不会创建业务表或 ES 索引。

12 服务地址、数据库客户端与数据卷

服务	Windows 主机地址	账号或凭据来源
MySQL 8.0	127.0.0.1:13306	库 campus_rag；用户 campus_dev；MYSQL_PASSWORD
Redis 7.4.2	127.0.0.1:6379	REDIS_PASSWORD
Elasticsearch + IK 8.11.4	http://127.0.0.1:9200	elastic；ELASTIC_PASSWORD
MinIO API / 控制台	http://127.0.0.1:9000 / http://127.0.0.1:9001	MINIO_ROOT_USER / MINIO_ROOT_PASSWORD
Nacos 2.3.2	http://127.0.0.1:8848/nacos；9848/9849	nacos；NACOS_ADMIN_PASSWORD
RocketMQ NameServer	127.0.0.1:9876	按需启用 Messaging
RocketMQ Broker	127.0.0.1:10911；另用 10909/10912	广播地址面向本机客户端
Ollama	http://127.0.0.1:11434	本地 API 不需要云账号登录

若已有数据库客户端，可新建 MySQL 连接，主机 127.0.0.1、端口 13306、用户 campus_dev，密码在自己的本机凭据文件中查看，选择 campus_rag 后测试连接。不要把负责人已有的 3306 实例改作本项目数据库。

同一 Compose 网络中的业务容器应连接 mysql:3306 等服务名和容器端口，不能用容器内部的 127.0.0.1 访问另一个容器。当前 RocketMQ 的本机公告地址也不能直接用作其他容器或其他电脑的连接地址。

结构与样例通过 database/mysql/migrations、database/mysql/seeds 共享；ES mapping 放在 database/elasticsearch/index-templates。目前这些目录尚无业务实现，不存在自动执行迁移的流程。

持久化卷由 Docker 引擎管理，不存放于 Git 仓库中的业务目录。日常停止使用 Stop-Services.ps1，保留卷和凭据；不要附加删除卷的选项。业务备份应通过数据库或对象存储的导出流程进行，不能把运行中的数据盘随意复制给队员。

13 Ollama、嵌入模型与重排模型安装

1. 从 Ollama 官方 Windows 页面安装，安装器通常会启动托盘后台服务。[S11] 使用 ollama --version、ollama list 检查工具；本地 API 不要求登录云账号。[S12]
2. 确定模型目录。项目期望 BGE 和重排权重在 .local/models，生成模型在 .local/ollama-models。若 Ollama 已在后台运行，后来设置客户端环境变量不会改变服务端模型目录。
3. 如需使用项目指定的生成模型目录，先由本人从托盘正常退出已有 Ollama，再运行项目脚本；不要结束不明进程。已有服务继续运行时也可复用，但模型可能仍下载到其原目录。
4. 在仓库根目录预建检查目录，再执行完整模型准备。首次下载可能占用较长时间和较多磁盘空间。

```
New-Item -ItemType Directory -Path '.setup\logs' -Force | Out-Null
.\environment\Prepare-Models.ps1
```

脚本内部顺序：安装固定 CPU 模型依赖 → 下载固定提交的 BGE 与重排权重并核验 → 启动或复用 Ollama → 拉取 qwen3:1.7b 并核对 digest → 嵌入、重排、生成验收。

```
# 已完成准备后，只重新验收或启动 API
.\environment\Test-Models.ps1
ollama list
ollama ps
Invoke-RestMethod -Uri 'http://127.0.0.1:11434/api/version'
```

成功标准：嵌入为 1024 维，中文相关材料得分高于无关材料；重排符合相关性；生成能回答构造材料问题并带引用标记。该结果不是完整业务平台验收。

Ollama 默认个人模型目录通常为 %USERPROFILE%\.ollama\models，本文使用项目目录仅为当前脚本约定。团队如改用统一 API 或 vLLM，须另行提供地址、鉴权和预算；目前没有可直接使用的共享地址。

14 模型配置、依赖锁与内存控制

用途	当前配置
中文嵌入	BAAI/bge-large-zh-v1.5，输出 1024 维；.local/models/bge-large-zh-v1.5
重排	BAAI/bge-reranker-base；.local/models/bge-reranker-base
本地生成	Ollama qwen3:1.7b；.local/ollama-models
参数参考	environment/model-env.example，只有无密钥参数
生成请求约定	keep_alive=0；options.num_ctx=2048；小批量调用

BAAI 官方 large 中文模型输出 1024 维，原手册的 768 维不适用于本模型。[S13] 业务 ES dense_vector.dims 必须与实际输出一致；不要截断向量凑成 768。重排模型用于候选排序，不能代替向量模型。

```
# 先用项目脚本启动 API，再进行一个最小生成请求
$body = @{
    model = 'qwen3:1.7b'
  prompt = '请用一句中文说明你已准备好。'
    stream = $false
    think = $false
    keep_alive = 0
    options = @{ num_ctx = 2048; num_predict = 120 }
} | ConvertTo-Json -Depth 4
$result = Invoke-RestMethod -Method Post `
    -Uri 'http://127.0.0.1:11434/api/generate' `
    -ContentType 'application/json; charset=utf-8' `
    -Body ([Text.Encoding]::UTF8.GetBytes($body))
$result.response
```

该请求仅验证模型 API，不验证检索与引用。keep_alive=0 要求请求后卸载生成模型，但其他模型或托盘服务仍可能占用内存。

PyTorch wheel 固定 Python 3.11 Windows x64 和 SHA256；同名 Qwen 标签变化时脚本会要求复核。安装失败不要绕过校验或随意升级整套依赖，先核对网络、架构、锁文件和发布来源。

15 工具、基础服务与模型分层验收

层次	检查方法	通过标准
工具与 SDK	Check-Environment.ps1；Test-PythonDependencies.py	使用正确解释器与工具，SDK 导入及离线调用通过
默认基础服务	Test-Services.ps1	五项服务 healthy，认证和 IK 分词通过
消息组件	Test-Services.ps1 -Messaging	七项服务检查及 Broker 注册通过
模型	Test-Models.ps1	1024 维、相关性、重排与生成检查通过
Java 结构	根目录 mvn -o -B validate	根 POM 与六个模块共七个项目结构有效
业务平台	见 18-19 节	待实现，不能用上述结果替代

```
.\environment\Check-Environment.ps1
.\environment\Test-Services.ps1
# 使用消息队列时，另行检查：
# .\environment\Test-Services.ps1 -Messaging
.\environment\Test-Models.ps1
mvn -o -B validate
mvn -o -B -pl backend/search-service -am validate
```

工具全量检查还包含 GitHub CLI 和 VS Code 的 Java/Python/Pylance/Vue 扩展。可按角色理解缺项：例如前端成员尚未安装本地模型时，不应把模型缺项描述成 MySQL 故障。需要完整验收时再补齐全部工具。

Check-Environment.ps1 -RequireServices 检查默认五项；七项仍需单独 Test-Services.ps1 -Messaging。不要只看端口打开就认定认证、分词和 Broker 注册都正确。

负责人已有检查结果仅证明当时本机环境。新成员需独立运行并保存自己的结果，报告放 .setup/logs，不上传公开仓库。

16 Java、Python 与前端当前可做的配置

模块	当前可做	尚未具备
Java 后端	安装 Java17/Maven，核对根 POM 与六个模块，执行 validate	Spring 版本组合、启动类、控制器、业务依赖与 Actuator
Python RAG	根 .venv、SDK 锁、模型、服务参数、目录职责	pyproject.toml、业务 API、真实 MCP 工具与问答链路
Vue 前端	Node/npm、编辑器、页面与 API 目录规划	package.json、包锁、Vite 配置及应用入口

Java 子模块在 backend 下，父 POM 路径为 ../../pom.xml；artifactId 和 cn.campus.rag 包前缀不变。根目录没有 Maven Wrapper 时使用已安装的 Maven。

```
# 现阶段：校验聚合和模块引用
mvn -o -B validate
mvn -o -B -pl backend/doc-service -am validate
```

原手册的 Spring Boot 3.2.x 与 Spring Cloud Alibaba 2022.x 组合不可直接照搬；应按官方兼容关系和实际 POM 统一选型。[S14] 不能把普通 Maven 骨架打包成功等同于业务服务能启动。

Python 根 .venv 不再在 rag-runtime 内重复创建；requirements.txt 引用统一 SDK 锁。environment/.env.application 只提供配置来源，当前没有自动加载这些变量的业务程序。

原手册中的 uvicorn src.api:app、pip install -e .、npm install、npm run dev，以及业务 /actuator/health 都不能当成现成启动步骤。8000、8080、5173 只是原文常见示例，目前没有已确定的业务端口或访问网址。

17 日常启动、停止与负责人本机复用

负责人当前工作目录为 D:\campus_RAG_work，已有工具、虚拟环境、模型、凭据和 Docker 独立数据目录。不要重新执行首次安装的克隆、venv 创建、初始化新密码或数据卷重建。

```
# 负责人现有环境：沿用已配置的独立引擎启动入口
Set-Location -LiteralPath 'D:\campus_RAG_work'
. .\environment\Enter-DevEnvironment.ps1
.\environment\Start-Services.ps1 -Messaging
.\environment\Start-ModelService.ps1
.\environment\Test-Services.ps1 -Messaging
```

负责人本机保留的一键包装和编辑器配置可以继续使用，但不在 Git 仓库分发。若需要查看本机历史安装、登录及修复记录，可在本机查阅 environment/负责人本机说明.local.md；该文件不随克隆提供。

新队员复用本人普通 Desktop 引擎时，每天先从开始菜单打开 Desktop，进入仓库并加载适配后的开发终端，再使用本指南 11 节的 Common + Compose 路线，不直接调用负责人独立数据目录启动器。

```
# 两种路线结束工作时均可使用：
.\environment\Stop-Services.ps1
.\environment\Stop-ModelService.ps1
```

停止只针对本项目服务；Docker 命名卷和凭据保留。Ollama 停止脚本只停止本环境记录且身份核验匹配的进程，不关闭先前由托盘启动、未被记录的服务。

查看服务状态时，可在加载 Common 后使用 Get-CampusComposeArguments 构建参数，再执行 docker @composeArgs ps。排障日志先去除密码、令牌、个人资料后再分享。不要在共享终端输出完整展开的 Compose 配置。

18 业务工程实现后的最小联调顺序

本节是后续开发清单，当前不能直接运行。团队先落实模块负责人、接口字段、鉴权、业务端口、表结构、ES mapping 和版本状态规则。

1. Java：统一 Spring Boot / Cloud / Alibaba 依赖，补启动类、配置绑定和健康接口；确认 Nacos 注册、MySQL、Redis、MinIO、ES 连接及权限传递。
2. Python：实现 API 入口和应用配置加载，接入混合检索、过滤、重排、模型调用、引用与护栏；实现 MCP 工具 search_regulation、get_clause、list_versions、check_expiry 的真实契约。
3. 前端：初始化 Vue/Vite/TypeScript 并提交包锁，落实 /api 到网关的代理、登录、问答、文档管理和引用卡片。前端变量会进入浏览器，不能放数据库或 LLM 密钥。
4. 业务数据：将迁移放入 database/mysql/migrations，人工样例放 seeds/fixtures，索引定义放 index-templates；明确迁移执行方式，不能假设启动中间件会自动建业务表。
5. 最小联调按中间件 → Java gateway/auth/doc/search → Python RAG → frontend 推进；管理、评测与完整异步链路按任务接入。
6. 文档上传依赖异步消息时启用 RocketMQ。无消息队列的最小问答演示只能在已存在且有效的测试索引基础上设计，不等于上传链路已可用。

业务镜像及 Compose 后续放 deploy，当前 environment/docker-compose.yml 只有中间件，不包含 Java、Python 或前端应用服务。原手册的 logs search-service 和 up --build 启动应用不适用于当前 Compose。

Nginx、Kubernetes、Sentinel、Seata、SkyWalking、vLLM 属于后续可选增强，当前没有集成或可用地址。按原手册列出这些组件，不代表必须为首次环境准备逐一安装。

19 RAG、引用、版本与权限验收标准

以下标准来自原手册，并修正维度及目录；应在业务功能实现后验收，当前不宣称已通过。

验收项	操作与通过标准
上传与索引	用可公开的构造 PDF 上传，跟踪解析、切分、向量化与 ES 入库；原手册目标为两分钟内 doc_chunk 可检索，chunk_count 与记录一致，最终指标由团队落实
混合检索	同时覆盖关键词与向量召回，按版本和权限过滤；Top-5 应含有依据的相关条款，不能仅凭向量相似度确认正确
回答与引用	构造“国家奖学金申请条件”等问题；至少一条可核验引用，前端 [C:n] 能对应文件、条款位置和原文片段
版本与时效	覆盖 ACTIVE、SUPERSEDED、ABOLISHED 及生效/失效日期；废止制度不被当作有效依据
权限	学生调用管理员接口应返回 403；检索结果不得越过知识库及文档授权范围
护栏	无 citations、低置信、敏感材料、Prompt 注入等按规则拒答、脱敏或降级
故障降级	Python 或 LLM 不可用时，Java 按设计降级为纯检索或明确错误，不应只返回不可解释的 500
评测	固定公开构造测试集，记录 Top-5、答案正确性、引用一致性、过期命中率及延迟等指标

ES 向量字段使用 dense_vector，维度为 1024。新增评测和权限用例前先确定接口和数据契约；当前不存在原手册中的 eval/run_eval.py、deploy/scripts/version_filter_test.sh 或 permission_test.sh，不能直接复制命令执行。

提交与演示前检查：构造材料可公开；无真实凭据与个人信息；回答引用能展开；版本与权限测试覆盖；失败和降级行为可复现。环境检查和模型构造问答成功不替代这些业务验收。

20 环境与安装问题排查

现象	先检查与处理
命令找不到 / 版本不对	重开终端；Get-Command 检查实际位置；核对 Enter 脚本是否覆盖 JAVA_HOME/MAVEN_HOME；Python 用 .venv 完整路径
.ps1 或 npm.ps1 被阻止	npm 用 npm.cmd；设备策略允许时只调整当前进程策略；受管策略由管理员处理
Docker 有 Client 没 Server	打开 Desktop，等待引擎；核对 WSL2、虚拟化、Linux containers、context 与命名管道
desktop-linux 不存在	新克隆的 CLI 配置不带 context；按 09 节在项目 CLI 配置中创建，不复制别人登录配置
Compose 插件不存在	核对 Initialize-ContainerClient 的候选目录与实际 Docker 安装位置，确认 docker compose version
容器或数据看似丢失	先检查是否切换 Desktop 配置、数据目录或 context；不要删除旧盘或重建卷
端口被占用	用 Get-NetTCPConnection 查看相关本机监听；MySQL 默认 13306，不干扰原 3306；其他端口需同步修改 Compose 与应用参数
账号认证失败	确认 .env 对应当前命名卷；仅改文件不会改数据库内密码；Nacos 用启动流程生成的密码
pip 或模型安装失败	核对 Python3.11 x64、锁文件、网络与官方来源；不要绕过 SHA256/digest；pip check 确认依赖一致
模型下载到非项目目录	已运行的 Ollama 不会采纳新客户端变量；由本人正常退出托盘后再用项目入口启动
模型内存紧张 / 响应慢	CPU 使用小批量与低上下文；生成后 keep_alive=0；非必须时停止消息组件/其他服务，避免并行占用
模型报告目录缺失	在仓库根目录先创建 .setup/logs；特别是复用已有 Ollama API 时

每次排障只处理已确认的组件。共享日志前去掉凭据和个人信息；读取状态即可定位时，不要重置密码、重建数据卷或结束未知进程。

21 业务链路问题与团队协作

以下业务现象只在对应功能实现后排查：

现象	排查方向
向量召回为空	查询向量与索引均为 1024 维；核对实际写入、mapping、过滤条件，修正后按迁移方案 reindex
关键词有效但向量差	编号、日期等精确匹配保留关键词；采用混合召回与融合，勿仅依赖向量
引用与原文不符	核对切分、citation 标记映射、提示词约束和忠实性校验；不通过则拒答
废止制度仍被回答	检查版本状态、生效/失效时间与检索过滤是否贯穿全链路
上传后没有索引	核对消息启用、消费、失败重试与死信；再查解析、嵌入和 ES 写入
LLM 超时或不稳定	核对服务可用性、并发和超时；按已实现设计熔断、拒答或降级
IK 切碎制度编号	审查分析器与自定义词典，并用真实字段分析测试确认
引用卡片不显示	检查前后端 citations 契约、[C:n] 解析和权限允许的原文链接
CORS / 代理异常	核对实际网关地址、Vite/Nginx /api 代理、流式连接与 WebSocket 头

协作时用本人 GitHub 账号。负责人通过 Settings → Collaborators → Add people 邀请，队员接受后才能向原仓库写入；没有写入权限可 Fork 后提跨仓库 PR。[S15]

```
# 已有写入权限时，在干净工作区同步并新建任务分支
git switch main
git pull --ff-only origin main
git switch -c feat/your-task
# 修改和检查后，只添加实际相关文件
git add 'README.md'
git commit -m 'Describe this change'
git push -u origin feat/your-task
```

在 GitHub 向原仓库 main 提交 PR，说明改动和验证结果，经团队审查后合并。示例 README.md 应换为本次真正修改的文件。结构、接口、依赖、配置或目录变动时同步更新相应说明。

22 原手册修订对照

原文或旧做法	本指南采用的当前约定
仓库地址占位符	ZhiShan717/campus-rag-platform 的真实 HTTPS 地址
六个服务在根目录	backend 下六个服务，根 POM 仍负责聚合
BGE large 768 维	1024 维，并与 ES mapping 一致
原 Spring 版本组合直接使用	业务初始化前按官方兼容关系统一，当前未选定
rag-runtime 内建 .venv、pip install -e .	根 .venv，按 SDK 锁文件安装；无对应 pyproject.toml
直接运行 uvicorn / npm run dev	当前入口不存在，作为后续工程实现内容
copy .env.example、deploy/docker-compose.yml	初始化脚本生成凭据；environment/docker-compose.yml
默认主机 MySQL 3306	主机 13306；容器内 mysql:3306
完整 Compose 包含应用	当前只有基础服务，业务部署仍待实现
克隆后复用负责人全部启动入口	新队员先适配工具与 Docker；个人辅助文件不分发

23 官方安装资料与参考

资料核对日期为 2026-10-23。本文命令依据原手册、当前仓库和以下官方资料综合编写；本机实际状态以现场检查为准。引用编号对应正文安装步骤，PDF 中的资料名称可点击打开。

编号	官方资料
S01	Docker Desktop Windows 安装与要求
S02	微软 WSL 安装
S03	微软 WSL 命令
S04	Git for Windows
S05	VS Code Windows 安装
S06	Temurin Windows 安装
S07	Maven 安装；3.9.9 官方归档
S08	Python 3.11 Windows 使用；3.11.9 兼容基线
S09	Node.js 下载
S10	Docker context create
S11	Ollama Windows；Windows 下载
S12	Ollama CLI；本地 API 认证说明
S13	BGE large 中文模型卡；重排 base 模型卡
S14	Spring Cloud Alibaba 官方项目与版本资料
S15	GitHub 邀请个人仓库协作者

原始文件：environment/docs/校园级RAG知识库平台环境安装与首次运行手册.pdf。仓库 POM、Compose、模型参数及锁文件是当前实现依据；environment/SOURCES.md 记录镜像摘要和替代来源。